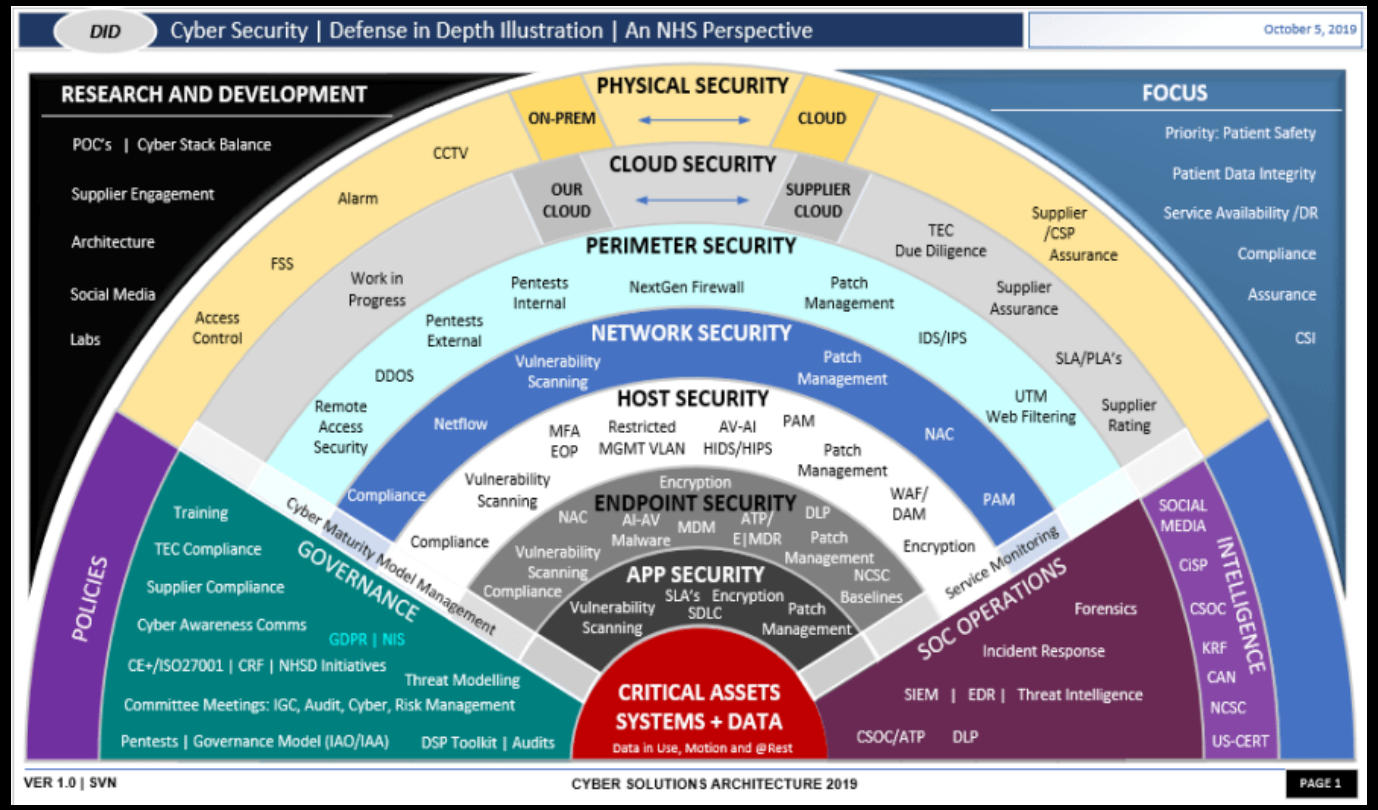
Green Pace

Security Policy Presentation

Developer: *Kimani Muhammad*

OVERVIEW: DEFENSE IN DEPTH

The Green Pace security policy defines the core security principles, the Triple A standards, and data encryption standards. This policy was needed to provide a consistent approach to implementing security layers and will be used as a blueprint for all development to support the implementation of defense-in-depth.



Green Pace

THREATS MATRIX

Likely

- String Correctness (STD-003-CPP)
- Resource Management (STD-008-CPP)

These issues occur frequently in C/C++ applications and often result in memory corruption, application instability, or denial-of-service attacks.

Priority

- Data Type Validation (STD-001-CPP)
- SQL Injection Prevention (STD-004-CPP)
- Memory Protection (STD-005-CPP)
- Secure Logging (STD-010-CPP)

These vulnerabilities can directly expose sensitive data, allow unauthorized access, or lead to system compromise and should be remediated immediately.

Low priority

- Data Value Validation (STD-002-CPP)
- Exception Handling (STD-007-CPP)

These primarily affect application integrity, reliability, and maintainability. They generally present a lower security impact than critical vulnerabilities.

Unlikely

- Assertions (STD-006-CPP)
- Integer Overflow Protection (STD-009-CPP)

These vulnerabilities are unlikely to but can still contribute to application instability, incorrect behavior, and potential security weaknesses if left unaddressed.

Green Pace

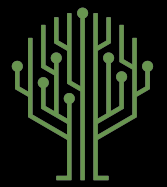
10 PRINCIPLES

Coding Principles	Coding Standards
Validate Input Data	Data Type, String Correctness, SQL Injectio, String Correctness, Data Value
Heed Compiler Warnings	Exceptions, Assertions
Architect and Design for Security Policies	Resource Management, Exceptions, SQL Injection
Keep It Simple	Resource Management, String Correctness
Default Deny	Secure Logging
Adhere to the Principle of Least Privilege	Secure Logging
Sanitize Data Sent to Other Systems	Secure Logging, SQL Injection
Practice Defense in Depth	Memory Protection, Resource Management, Exceptions, Data Value
Use Effective Quality Assurance Techniques	Assertions, Exceptions
Adopt a Secure Coding Standard	Secure Logging, Exceptions, Memory Protection, SQL Injection, String Correctness, Data Value, Data Type

Green Pace

CODING STANDARDS

	Critical	High	Medium
Coding Standards	SQL Injection Prevention (STD-004-CPP), Memory Protection (STD-005-CPP), Resource Management (STD-008-CPP), Data Value Validation (STD-002-CPP), and String Correctness (STD-003-CPP)	Data Type Validation (STD-001-CPP), Exception Handling (STD-007-CPP), and Secure Logging (STD-010-CPP)	Assertions (STD-006-CPP) and Integer Overflow Protection (STD-009-CPP)
Explanation	These standards have the highest priority because they can directly impact the confidentiality, integrity, and availability of the system. SQL injection vulnerabilities can allow attackers to access or modify sensitive database information. Memory protection and string correctness issues can lead to memory corruption, buffer overflows, application crashes, or remote code execution. Resource management failures can cause resource leaks that eventually create denial-of-service.	These issues can contribute to security weaknesses and operational failures. Improper input validation can allow invalid or malicious data to enter the application, while poor exception handling can hide errors and make troubleshooting difficult. Insecure logging can expose sensitive information such as credentials or personal data to unauthorized users.	These issues affect application stability and correctness rather than directly exposing sensitive data. Assertions may be disabled in production environments, making them unreliable for critical validation. Integer overflows can produce incorrect results and can also create security vulnerabilities.



ENCRYPTION POLICIES

- At Rest: Supported by **Memory Protection (STD-005-CPP)** and **Secure Logging (STD-010-CPP)**. Memory protection helps prevent unauthorized access to stored data, while secure logging ensures sensitive information is not exposed through logs. These standards complement AES-256 encryption by reducing the risk of stored data being compromised.

- In Flight: Supported by **Data Type Validation (STD-001-CPP)**, **Data Value Validation (STD-002-CPP)**, and **SQL Injection Prevention (STD-004-CPP)**. Validating and sanitizing data before transmission helps ensure that malicious or malformed data is not sent across the network. SQL injection prevention further protects data integrity by preventing attackers from manipulating transmitted input that could affect backend systems.

- In Use: Supported by **Memory Protection (STD-005-CPP)**, **Resource Management (STD-008-CPP)**, and **Integer Overflow Protection (STD-009-CPP)**. These standards help protect sensitive data while it is actively being processed in memory. Memory protection prevents unauthorized access, resource management reduces exposure caused by leaked resources, and integer overflow protection helps prevent unexpected behavior that could expose or corrupt sensitive data.

Green Pace

TRIPLE-A POLICIES

- **Authentication:** Supported by **Data Type Validation (STD-001-CPP)**, **Data Value Validation (STD-002-CPP)**, and **Assertions (STD-006-CPP)**. Data type and value validation help ensure that login credentials are entered in the correct format and within expected ranges before they are processed. Assertions and runtime validation help us verify that the required authentication and user information are present before permitting access. These standards work together to help ensure that only valid users can successfully authenticate.

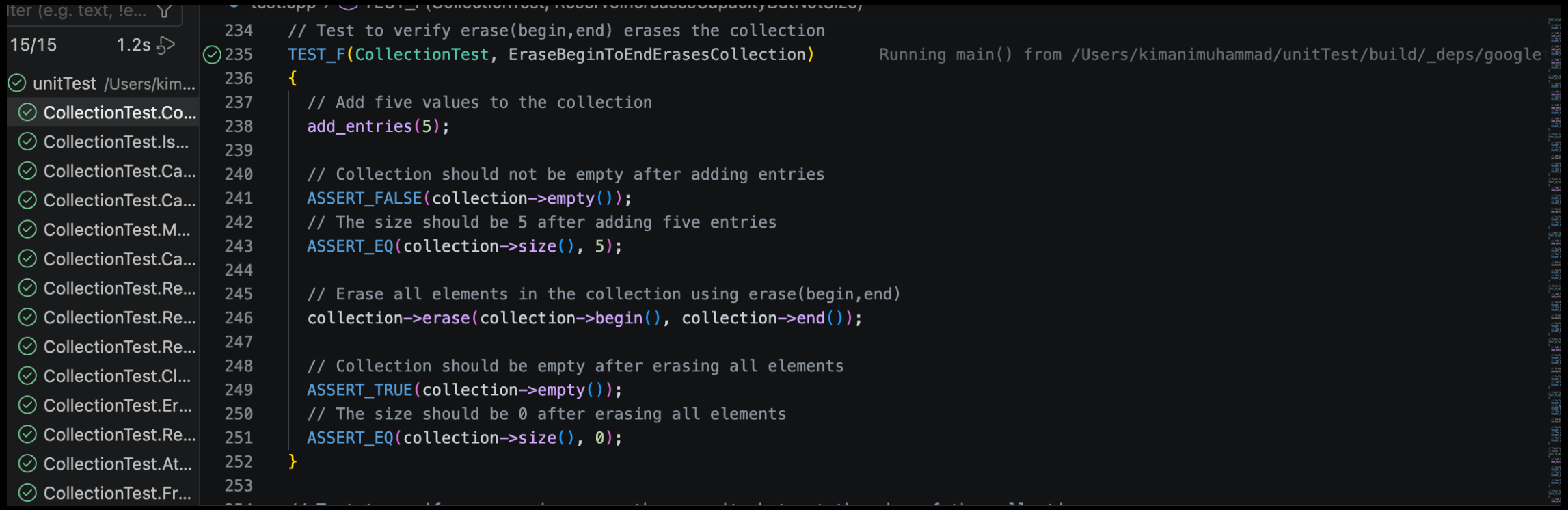
- **Authorization:** Supported by **Memory Protection (STD-005-CPP)** and **Resource Management (STD-008-CPP)**. Memory protection helps prevent unauthorized access to protected resources through vulnerabilities caused by memory corruption. Resource management ensures that system resources are properly controlled and released from memory. These two standards support the principle of least privilege by helping enforce secure access controls and preventing unauthorized actions from being taken.

- **Accounting:** Supported by **Secure Logging (STD-010-CPP)** and **Exception Handling (STD-007-CPP)**. Secure logging creates a record of each user login, file access, permission change, and database modification without exposing sensitive information. Exception handling ensures errors and security events are properly recorded and monitored instead of being silently ignored. These standard work together to provide accountability for system changes and support auditing.

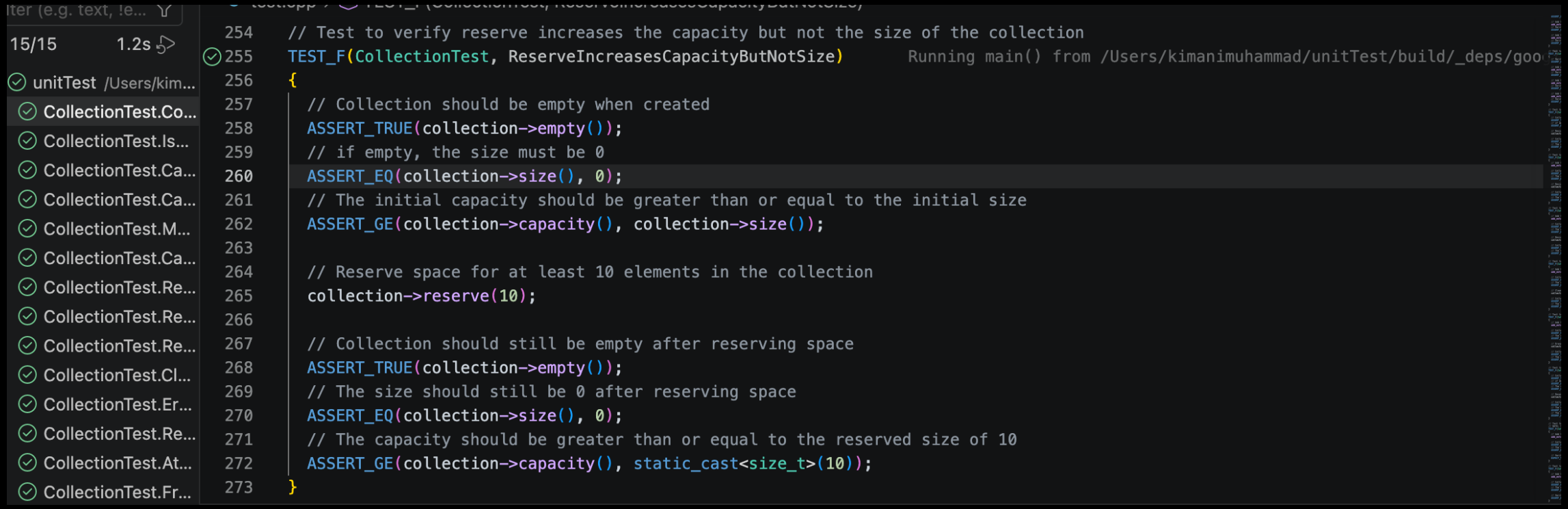
Green Pace

Unit Testing

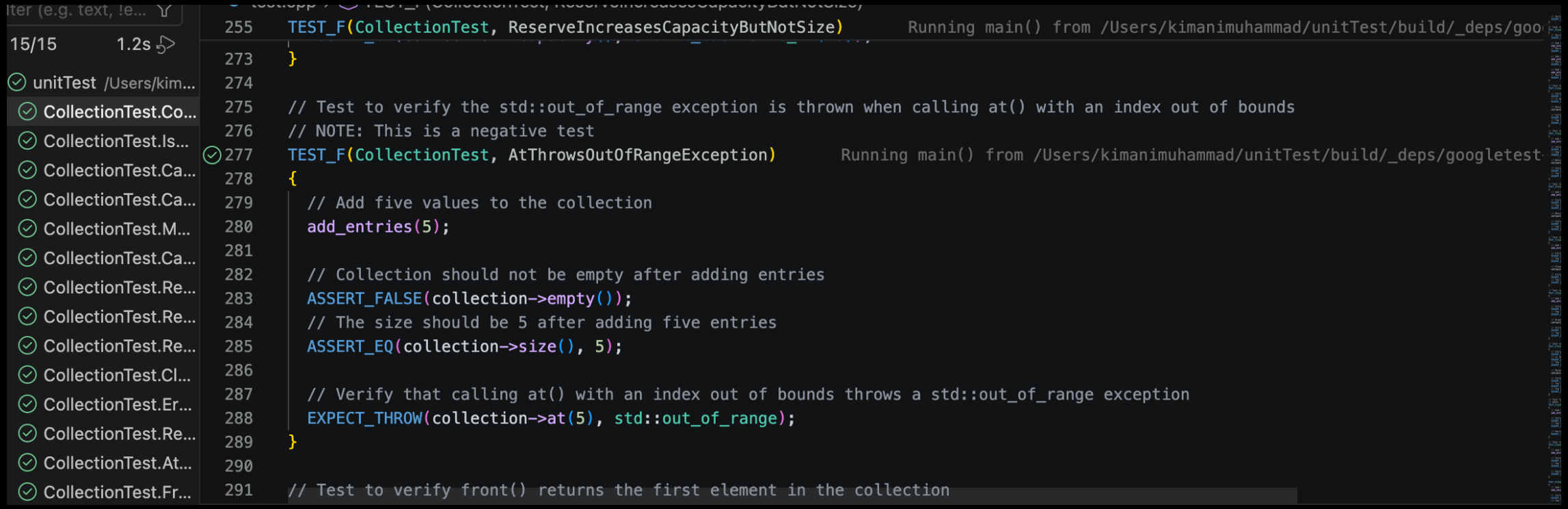
EraseBeginToEndErasesCollection



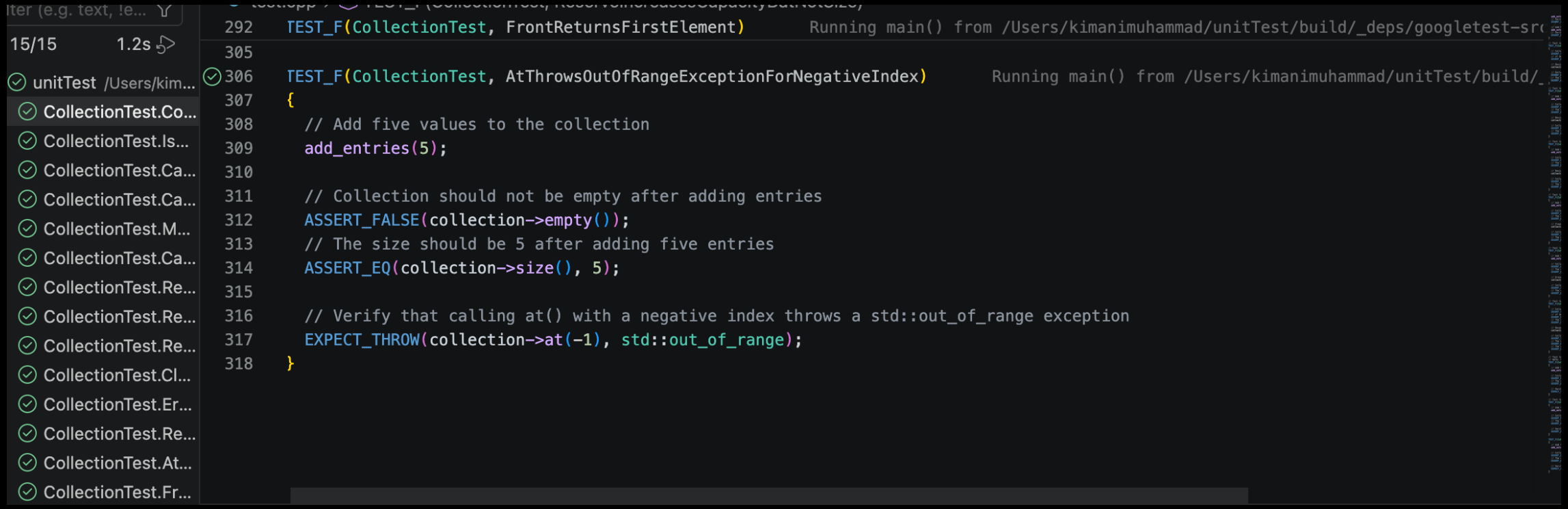
ReserveIncreasesCapacityButNotSize



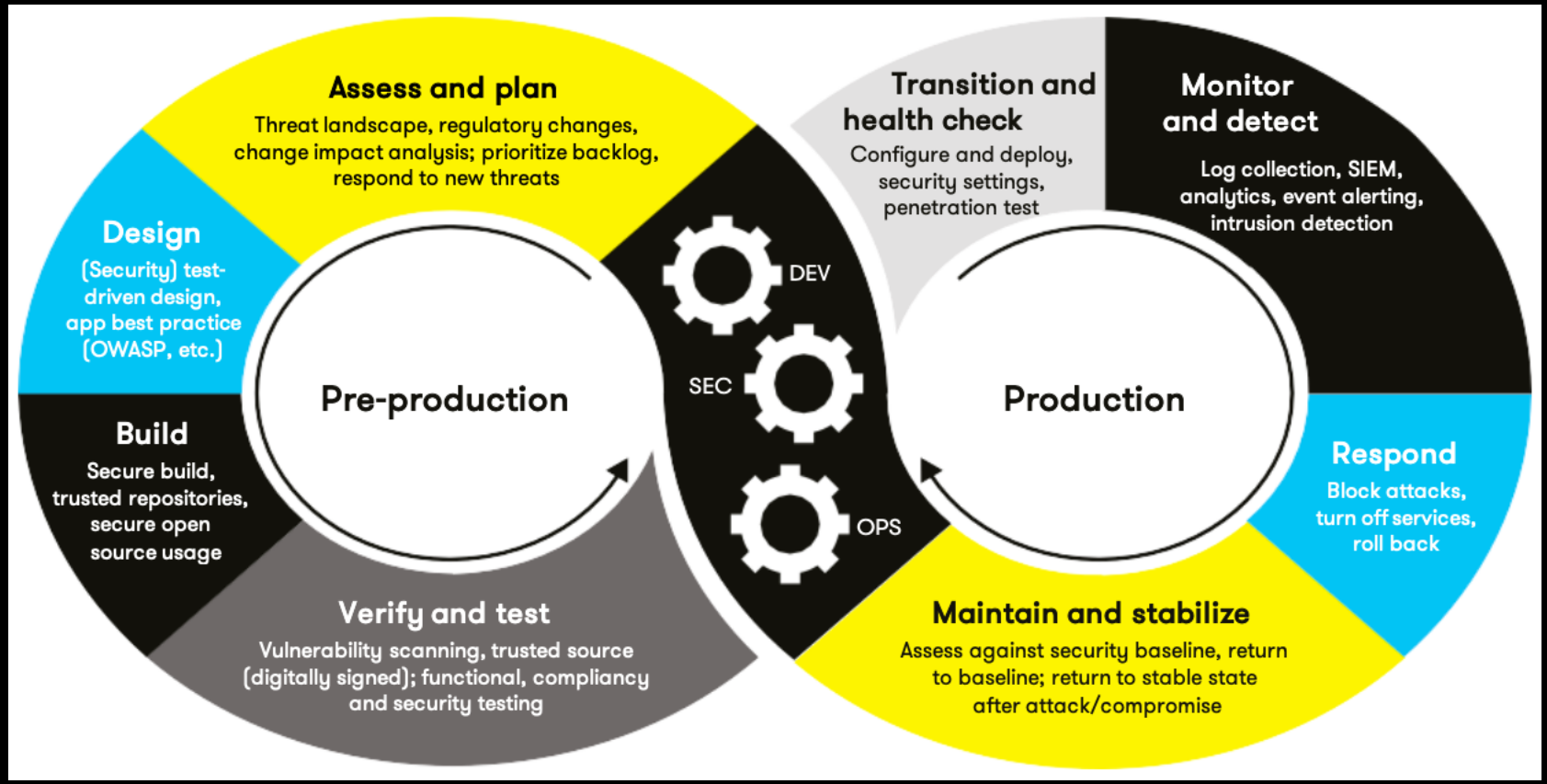
AtThrowsOutOfRangeException



AtThrowsOutOfRangeExceptionForNegativeIndex



AUTOMATION SUMMARY



Green Pace

TOOLS

- In the DevSecOps pipeline, security is implemented in each step of the pipeline to ensure security is considered at every step. The pipeline is made up of 4 pre-production phases, Assess and Plan, Design, Build, and Verification/Testing, and 4 production phases, Transition and Health check, Monitor and detect, Respond, and Maintain and stabilize. These phases continue to loop in a cycle as new development occurs.
- External tools used in the pre-production phase include: **Jira** used to track and plan development, **static testing tools** used to verify and test code
- External tools used in the production phase include: **log compiler** used to investigate interaction with the system during the monitor and protect phase.

Green Pace

RISKS AND BENEFITS

Act Now	Act Later
• Considering security immediately • Protects the system from known vulnerabilities • Act immediately whenever possible to best protect the system.	• Waiting until the end to consider security causes technical debt. • More vulnerabilities will be present, potentially exposing the system. • Never leave security until the end.

Green Pace

RECOMMENDATIONS

- **Log retention policy** - does not define log retention periods which can have legal ramifications.
- **Data classification policy** - does not define different classifications of data such as public, internal, confidential, and restricted.
- **Third-party and vendor policy** - does not address risks introduced by contractors, cloud providers, software vendors, or third-party services.

Green Pace

CONCLUSIONS

- To prevent future problems, one standard that should be adopted is **peer code reviews** when new pull requests are made before merging the changes to reduce or prevent new bugs being introduced.
- Another standard that should be adopted is a **Zero Trust policy** to make sure the system can't be compromised by stolen credentials since they would still need to be verified and approved.

Green Pace

REFERENCES

- Check Point Software. (2019, November 21). What is Zero Trust Security? [Video]. YouTube. https://youtu.be/1D5mg9an19o
- Cppcheck Team. (n.d.). Cppcheck: A tool for static C/C++ code analysis. https://cppcheck.sourceforge.io/
- David Lee. (2020, April 14). Authentication vs Authorization [Video]. YouTube. https://youtu.be/4shufXWd1XM
- Muhammad, K. (2026). Green Pace secure development policy [Unpublished manuscript]. Southern New Hampshire University.
- TechBeacon. (2024, January 17). 6 DevSecOps best practices: Automate early, often. Internet Archive. https://web.archive.org/web/20240117143629/https://techbeacon.com/security/6-devsecops-best-practices-automate-early-often

Green Pace